Graphs I

Representations, BFS, DFS, components, grids and bipartite check

Learning goals

- Read a graph from input and store it as an **adjacency list**
- Run **BFS** to get shortest paths in unweighted graphs, and rebuild the path
- Run **DFS** (recursive and iterative) and count connected components
- Treat a **grid** as an implicit graph (flood fill, shortest path in a maze)
- Test whether a graph is **bipartite** by 2-colouring
- Know the cost: everything today is O(n + m)

Why graphs?

Anything with "things" and "connections between pairs of things":

- cities and roads, computers and cables, people and friendships
- cells of a maze and the moves between them
- states of a puzzle and the moves between them

Today's question types:

- Can I get from A to B? How many steps at least?
- How many separate pieces are there?
- Can I split the vertices into two groups with all edges between groups?

Vocabulary

- Graph G = (V, E): n = |V| vertices, m = |E| edges
- **Undirected** edge {u, v} vs **directed** edge u → v
- **Weighted** edges carry a cost (next week)
- **Degree**: number of incident edges; Σ deg = 2m (undirected)
- **Path**, **cycle**, **connected**, **component**
- **Tree**: connected, n − 1 edges, no cycle (Week 20)
- Simple graph: no self-loops, no multi-edges — **never assume it** unless stated

Representations

	Adjacency matrix	Adjacency list	Edge list
Memory	$O(n^2)$	$O(n + m)$	$O(m)$
Is u–v an edge?	$O(1)$	$O(\deg u)$	$O(m)$
Iterate neighbours	$O(n)$	$O(\deg u)$	$O(m)$
Typical use	$n \le \sim 2000$, Floyd-Warshall	**almost everything**	Kruskal, Bellman-Ford

With $n = 10^5$ a matrix needs 10^{10} cells — impossible. Default to the **list**.

Reading a graph

```
int n, m;
cin >> n >> m;
vector<vector<int>> adj(n + 1);          // 1-indexed vertices
for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;
    adj[a].push_back(b);
    adj[b].push_back(a);                 // omit for a directed graph
}
```

Check the statement: 0- or 1-indexed? Directed? Multi-edges? Self-loops?

Breadth-first search: idea

Explore in **layers** of distance 0, 1, 2, … using a FIFO queue.

```
      1              layer 0: {1}
    / | \
   2  3  4           layer 1: {2, 3, 4}
      |   \
      6    5         layer 2: {6, 5}
```

Claim: when a vertex is first discovered, its `dist` is the **shortest** number of edges from the source.
Why: the queue always holds vertices of layer d followed by layer d+1, never anything else.

BFS code

```
vector<int> dist(n + 1, -1), par(n + 1, 0);
queue<int> q;
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u])
        if (dist[v] == -1) {          // mark when PUSHED, not when popped
            dist[v] = dist[u] + 1;
            par[v] = u;
            q.push(v);
        }
}
```

Each vertex enters the queue once, each edge is examined twice → **O(n + m)**.

Rebuilding the path (CSES 1667 Message Route)

```
if (dist[n] == -1) { cout << "IMPOSSIBLE\n"; return 0; }
vector<int> path;
for (int v = n; v != 1; v = par[v]) path.push_back(v);
path.push_back(1);
reverse(path.begin(), path.end());
```

Parent pointers form a **BFS tree** rooted at the source; following them back gives a shortest path.

Depth-first search: idea

Go as deep as possible, backtrack when stuck.

```
vector<bool> vis(n + 1, false);
void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}
```

- Also O(n + m)
- Does **not** give shortest paths
- Gives structure: entry/exit times, DFS tree, back edges (cycles), subtree sizes (Weeks 20–22)

Recursion depth danger

A path graph 1–2–3–…–10^5 makes recursive DFS go 10^5 calls deep.

- Codeforces: 256 MB stack — usually fine
- Some judges (and your local machine, 8 MB default): **stack overflow → RE**

Iterative version with an explicit stack:

```
vector<int> st = {s}; vis[s] = true;
while (!st.empty()) {
    int u = st.back(); st.pop_back();
    for (int v : adj[u])
        if (!vis[v]) { vis[v] = true; st.push_back(v); }
}
```

(Visits the same set; order differs from recursive DFS.)

Connected components

Loop over all vertices; each unvisited one starts a new component.

```
int comps = 0;
for (int v = 1; v <= n; v++)
    if (!vis[v]) { comps++; dfs(v); }
```

CSES 1666 Building Roads: answer = comps − 1 new roads; connect one representative of each component to the next.

Total work is still O(n + m): every vertex and edge is handled once overall.

Grids are graphs

Cells = vertices, moves to the 4 neighbours = edges. No need to build `adj`.

```
const int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;   // bounds first!
    if (g[nr][nc] == '#' || seen[nr][nc]) continue;
    ...
}
```

8 directions (king moves) or knight moves: just change the arrays.

Flood fill: Counting Rooms (CSES 1192)

```
########        Count connected regions of '.'
#..#...#
####.#.#        → 3 rooms
#..#...#
########
```

For each unseen `.` cell: rooms++, BFS/DFS from it marking everything reachable.

O(n·m) total. Use BFS or iterative DFS — a 1000×1000 grid can make recursive DFS 10^6 deep.

Shortest path in a maze (CSES 1193 Labyrinth)

- BFS from `A`, `dist[r][c]`, stop when `B` is reached
- Store the **direction** used to enter each cell (`'U','D','L','R'`)
- Walk back from `B` using the stored directions, reverse the string

Multi-source BFS: push **all** sources with dist 0 at the start
(e.g. CSES 1194 Monsters: distance from the nearest monster to every cell).

Bipartite graphs

Bipartite ⇔ vertices can be coloured with 2 colours so every edge joins different colours ⇔ **no odd cycle**.

```
1 — 2          colours: 1→A, 2→B, 3→A, 4→B    bipartite ✓
|   |
4 — 3          triangle 1–2–3–1: impossible      ✗
```

Algorithm: BFS/DFS, give each new vertex the opposite colour of its parent; if an edge joins two equal colours → not bipartite.

Bipartite check code (CSES 1668 Building Teams)

```
vector<int> color(n + 1, 0);
for (int s = 1; s <= n; s++) {
    if (color[s]) continue;                 // every component!
    color[s] = 1;
    queue<int> q; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!color[v]) { color[v] = 3 - color[u]; q.push(v); }
            else if (color[v] == color[u]) { cout << "IMPOSSIBLE\n"; return 0; }
        }
    }
}
```

BFS vs DFS — which one?

Need	Use
Shortest path, unweighted	**BFS**
Components, reachability, flood fill	either
Bipartite check	either
Cycle detection in directed graph, topological order	**DFS** (Week 21)
Subtree info, entry/exit times	**DFS** (Weeks 20, 22)
Very deep graph, worried about stack	BFS or iterative DFS

Complexity

Operation	Time	Memory
Build adjacency list	O(n + m)	O(n + m)
BFS / DFS from one source	O(n + m)	O(n)
All components	O(n + m)	O(n)
Grid BFS / flood fill	O(R·C)	O(R·C)
Bipartite check	O(n + m)	O(n)

$n, m \le 2 \cdot 10^5$ is completely routine.

Common pitfalls

- Marking visited when **popped** instead of when pushed → duplicates, TLE
- Forgetting the reverse edge for an undirected graph
- Only searching from vertex 1 when the graph is disconnected
- Bounds check *after* indexing the grid → out-of-range access
- Recursive DFS on a 10^6-cell grid → stack overflow
- `vector<vector<int>> adj(n)` with 1-indexed input → off-by-one RE
- Using a matrix for n = 10^5

Practice problems

- CSES 1192 Counting Rooms · CSES 1193 Labyrinth
- CSES 1666 Building Roads · CSES 1667 Message Route
- CSES 1668 Building Teams · CSES 1669 Round Trip (find a cycle)
- CSES 1194 Monsters (multi-source BFS)
- CF 1037D Valid BFS? · CF 862B Mahmoud and Ehab and the bipartiteness
- CF 377A Maze · USACO 2016 Dec Silver "Moocast"

Summary

- Adjacency list is the default representation: O(n + m)
- **BFS** = layers = shortest paths in unweighted graphs; keep `par` for the path
- **DFS** = deep exploration; watch recursion depth
- Components: start a search from every unvisited vertex
- Grids: neighbours via direction arrays, no explicit graph needed
- Bipartite ⇔ 2-colourable ⇔ no odd cycle
- Next week: **weighted** shortest paths — Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS